

NGÔN NGỮ LẬP TRÌNH PHP

NGÔN NGỮ LẬP TRÌNH PHP

I. Giới thiệu về PHP

1. Giới thiệu

- + PHP (*Personal Home Page*) là một kịch bản trên Server (Server Script)

- + Là một môi trường lập trình cung cấp cho việc kết hợp HTML, ngôn ngữ kịch bản Script và các thành phần được viết trong các ngôn ngữ nhằm tạo ra một ứng dụng Internet mạnh mẽ và hoàn chỉnh.

- + PHP là một phần mềm mã nguồn mở (Open Source)

NGÔN NGỮ LẬP TRÌNH PHP

I. Giới thiệu về PHP

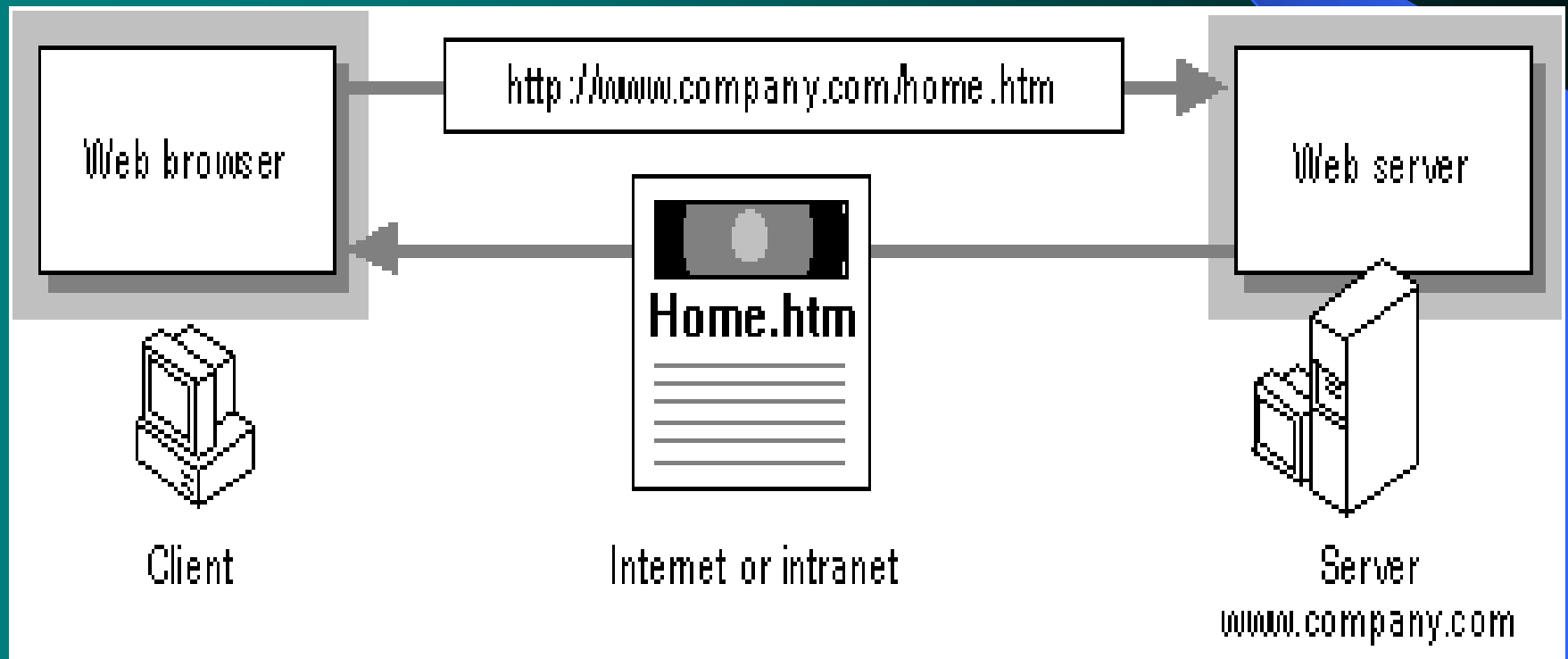
2. PHP file là gì?

- + File được tạo với phần mở rộng *.php.
- + Khi cần sửa đổi các file PHP ta chỉ cần ghi lại trên server thôi.
- + Vào những lần sau khi trang PHP được gọi, các Script trong file PHP tự động biên dịch lại.
- + Cấu trúc mã gần giống ngôn ngữ lập trình C

NGÔN NGỮ LẬP TRÌNH PHP

I. Giới thiệu về PHP

3. Cách thức làm việc của trang PHP + Trang HTML tĩnh:

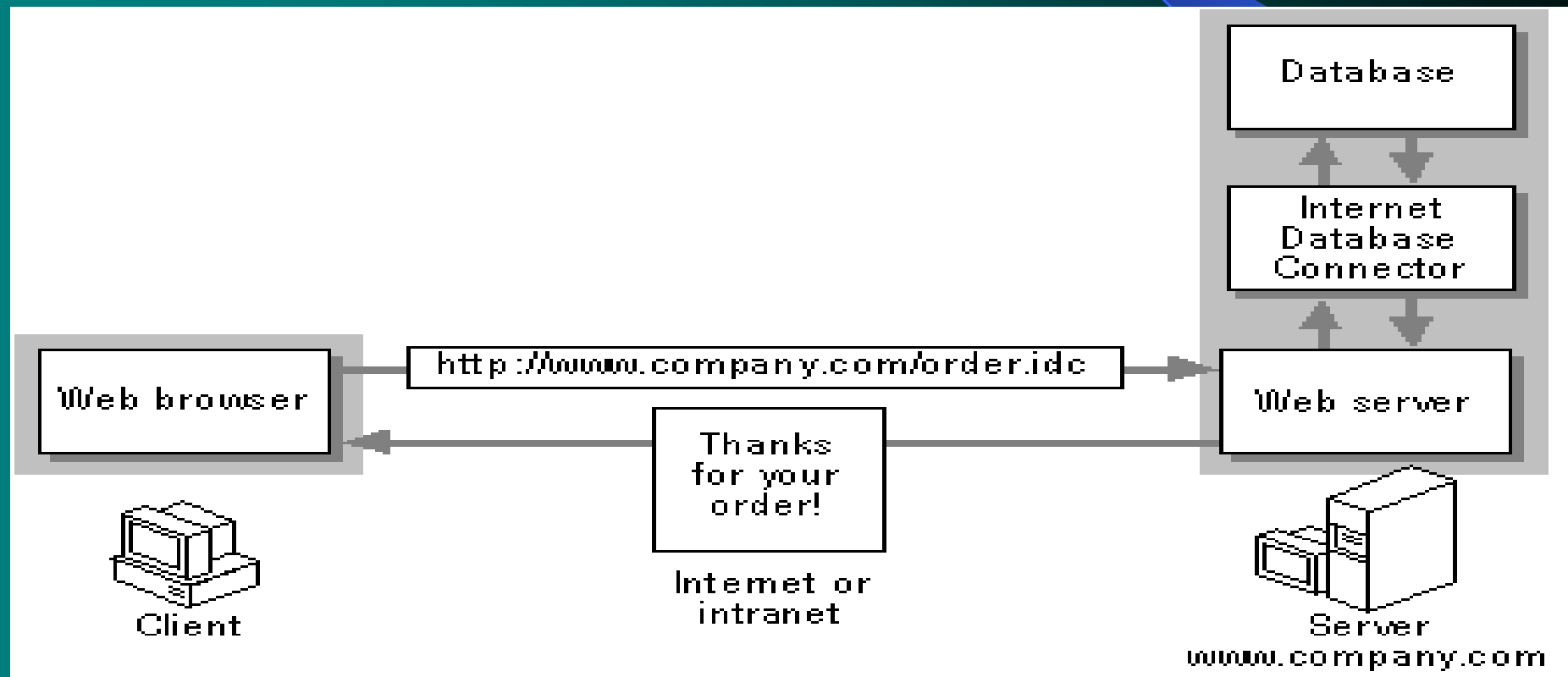


NGÔN NGỮ LẬP TRÌNH PHP

I. Giới thiệu về PHP

3. Cách thức làm việc của trang PHP

+ Trang PHP :



NGÔN NGỮ LẬP TRÌNH PHP

I. Giới thiệu về PHP

4. Web Server Apache thực thi trang PHP

- + Web về cơ bản thực sự là một hệ thống các yêu cầu (Request) và các đáp ứng (Response).
- + Webserver phản hồi lại các yêu cầu thông tin của Web Browser. Nó lắng nghe các yêu cầu đó từ phía Client trên một mạng sử dụng WWW.

NGÔN NGỮ LẬP TRÌNH PHP

II. Cách xây dựng Web bằng PHP

1. Nhúng PHP vào HTML

Có 4 cách để nhúng mã PHP vào trong HTML :

+ Dùng cặp thẻ `<? ..... ?>`

+ Dùng `<?php ....?>`

+ Dùng `<script language = "php" >`

.....

`</script>`

+ Dùng `<% ..... %>`

NGÔN NGỮ LẬP TRÌNH PHP

II. Cách xây dựng Web bằng PHP

1. Nhúng PHP vào HTML

Ví dụ 1: Lưu file sau lên đĩa với tên vd.php

```
<html>
```

```
  <head>
```

```
    <title>Testing page</title>
```

```
  </head>
```

```
  <body><?php echo "Hello you!"; ?>
```

```
  </body>
```

```
</html>
```

NGÔN NGỮ LẬP TRÌNH PHP

II. Cách xây dựng Web bằng PHP

2. Một số chú ý khi bắt đầu lập trình PHP

a. Lệnh echo :

Lệnh này dùng để xuất chuỗi văn bản về cho trình duyệt.

Ví dụ: Câu lệnh *echo "Hello, world!"*;
trình duyệt sẽ nhận được chuỗi văn bản Hello, world!.

Câu lệnh *echo 1+2*; sẽ trả về cho trình duyệt chuỗi văn bản 3.

Và câu lệnh *echo 5+2, "Hello world!"*; sẽ trả về trình duyệt chuỗi *7Hello world!*

NGÔN NGỮ LẬP TRÌNH PHP

II. Cách xây dựng Web bằng PHP

2. Một số chú ý khi bắt đầu lập trình PHP

b. Phân cách các lệnh bằng dấu chấm phẩy (;)

Tương tự như các ngôn ngữ lập trình khác, một câu lệnh của PHP được kết thúc bằng dấu chấm phẩy (;)

Ví dụ: `echo 1+2;` `echo "Hello, world!";`

c. Chú thích trong chương trình

- Chú thích 1 dòng được bắt đầu bằng // hoặc #, và những gì được ghi từ đó về sau là chú thích.

- Chú thích nhiều dòng được bắt đầu bằng / và kết thúc bằng */, những gì ở giữa là chú thích.*

NGÔN NGỮ LẬP TRÌNH PHP

II. Cách xây dựng Web bằng PHP

2. Một số chú ý khi bắt đầu lập trình PHP

c. Chú thích trong chương trình

Ví dụ:

```
<?php
```

```
//Đây là chú thích 1 dòng
```

```
#Đây cũng là chú thích 1 dòng
```

```
/* Đây là chú thích nhiều dòng Đoạn chương trình  
sau sẽ in ra chuỗi abc123 */
```

```
echo "abc123";
```

```
?>
```

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

PHP hỗ trợ 8 kiểu dữ liệu chính:

- + 4 kiểu dữ liệu vô hướng: boolean, integer, float (double), string.
- + 2 kiểu dữ liệu tổ hợp: array, object.
- + 2 kiểu dữ liệu đặc biệt : resource, NULL

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

+ *Kiểu Boolean* : mang giá trị *TRUE* (đúng) hoặc *FALSE* (sai).

Ví dụ: `<?php $a = TRUE; $b = FALSE;`

`//phép toán == kiểm tra xem 2 biểu thức`
`$c = (7==2);`

`//vì 7 khác 2 nên $c mang giá trị FALSE`

`$d = ("abc" == "abc"); //$d mang giá trị TRUE ?>`

`?>`

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

+ Kiểu Integer :

- Mang các giá trị số nguyên ..., -2, -1, 0, 1, 2,...
- Trên hầu hết các hệ thống, kiểu số nguyên có kích thước 32 bit, mang giá trị từ -2147483647 cho đến 2147483648.

Ví dụ:

```
<?php $a = 1234; $b = -123; $c = 0123;  
?>
```

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

+ *Kiểu Float (Double)* : là kiểu số thực, có thể mang bất cứ giá trị số thực nào. Trên hầu hết các hệ thống, kiểu số thực có kích thước 64 bit.

Ví dụ: <?php

```
$a = 1.234;
```

```
$b = 1.2e3; //= 1.2*10^3 = 1200
```

```
$c = 7E-10; //= 7*(10^-10) = 0.00000000007
```

```
$d = -1.23;
```

```
?>
```

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

+ **Kiểu String** : lưu giữ một chuỗi ký tự, mỗi ký tự có kích thước 1 byte. Nội dung string được đặt giữa hai dấu nháy, nháy đơn (') hoặc nháy kép (").

Ví dụ: <?php

`$a = 'Đây là 1 chuỗi được đặt giữa dấu nháy đơn';`

`$b = "Đây là 1 chuỗi được đặt giữa dấu nháy kép";`

`$c = 'Đây là 1 chuỗi được đặt giữa dấu nháy đơn với "vài dấu nháy kép ở giữa";`

`$d = "chuỗi đặt giữa dấu nháy kép với 'vài dấu nháy đơn '";`

`?>`

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

+ Nếu muốn sử dụng dấu nháy đơn trong một chuỗi được bọc bởi dấu nháy đơn, hoặc sử dụng dấu nháy kép đặt giữa chuỗi được bọc bởi dấu nháy kép thì để thêm ký tự \ (ký tự escape) ở phía trước.

Ví dụ: <?php \$a = 'Dấu \'nháy đơn\' ở giữa chuỗi';

// \$a mang giá trị: Dấu 'nháy đơn' ở giữa chuỗi

\$b = "Dấu \"nháy kép\" ở giữa chuỗi";

// \$b mang giá trị: Dấu "nháy kép" ở giữa chuỗi

\$c = "Dùng ký tự \\ ở giữa câu \\ thì sao?";

// \$c mang giá trị: Dùng ký tự \ ở giữa câu \ thì sao?

?>

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

- Ngoài ra, nếu để một biến vào giữa một chuỗi được bọc với dấu nháy kép, giá trị của biến sẽ được thay thế vào trong chuỗi.

Ví dụ: `<?php` `$a = 1;`
 `$b = 2;`
 `$c = 3;`
 `$d = "$a $b $c";`

// \$d sẽ mang giá trị là chuỗi "1 2 3"

`?>`

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

+ *Kiểu Array* : Array là một mảng gồm nhiều phần tử. Array được tạo qua lệnh Array.

Ví dụ:

```
<?php $a = Array(1,2,3);
```

```
    echo $a[0]; //in ra giá trị 1
```

```
    echo $a[2]; //in ra giá trị 3
```

```
    $a[1] = 5; //giờ đây $a = Array(1,5,3)
```

```
?>
```

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

- *Mảng còn có thể được tạo thành bởi các cặp (khoá, giá trị).*

Ví dụ:

```
<?php $a = Array(  
    "khoá 1" => "giá trị 1",  
    "khoá 2" => "giá trị 2",  
    "khoá 3" => "giá trị 3" );  
echo $a["khoá 1"]; //in ra: giá trị 1
```

● ?>

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

1. Kiểu dữ liệu :

+ *Kiểu NULL* : Đây là 1 giá trị đặc biệt, báo cho PHP biết rằng một biến nào đó chưa/không mang giá trị nào

Ví dụ: `<?php $a = 1; //$a mang giá trị 1`

`$a = NULL; //bây giờ $a không mang giá trị`

`$a = 2; //giờ đây $a mang giá trị 2`

`//hàm unset sẽ làm cho 1 biến có giá trị là NULL`

`unset($a); //lúc này $a lại là NULL`

`?>`

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

2. Biến trong PHP :

Một biến trong PHP được bắt đầu bằng ký tự \$ và đi theo ngay sau đó là tên của biến.

+ *Biến trong PHP phân biệt chữ hoa và chữ thường. \$Abc và \$abc là 2 biến hoàn toàn khác nhau.*

+ *Tên biến chỉ được bao gồm các ký tự chữ cái (a..z hoặc A...Z), chữ số (0...9) và ký tự gạch dưới (_); nhưng tên biến không được bắt đầu bằng ký tự gạch dưới hoặc chữ số. Ví dụ: \$a: biến có tên là a
\$abc123: biến có tên là abc123*

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

2. Biến trong PHP :

Ví dụ : Các tên biến sau là không hợp lệ :

`$_abc` Không hợp lệ! bắt đầu bằng ký tự gạch dưới

`$1abc` Không hợp lệ! bắt đầu bằng chữ số

`$nguyễn` Không hợp lệ! tên biến có ký tự đặc biệt (ẽ)

* *Tầm vực (scope) của biến* : là ngữ cảnh mà ở trong đó biến được định nghĩa.

Ví dụ: `<?php $a = 1; //tầm vực của biến $a từ đây`

`include 'b.php'; //trái dài tới bên trong file b.php`

`//tới cuối file vẫn còn hợp lệ`

`?>`

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

3. Biểu thức :

+ *Biểu thức là nền tảng quan trọng của PHP. Hầu như mọi thứ bạn ghi trong file php đều là biểu thức. Nói một cách đơn giản, bất cứ cái gì mang 1 giá trị nào đó đều có thể là 1 biểu thức. Ta xét câu lệnh đơn giản sau:*

$\$a = 5;$

$\$b = \$a;$

Tuy nhiên biểu thức trong PHP có thể phức tạp hơn

Ví dụ: *$\$a = 1; \$b = 2; \$c = 3; \$d = \$a + \$b + \$c;$*

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

4. Toán tử :

** Toán tử kết hợp các giá trị hoặc biểu thức lại với nhau và tạo ra một giá trị mới.*

Ví dụ: Trong biểu thức $1+2$ thì “+” là toán tử kết hợp hai giá trị 1 và 2 lại với nhau tạo ra giá trị mới là 3.

** Các toán tử trong PHP được chia thành 3 nhóm:*

- Các toán tử áp dụng trên 1 giá trị, ví dụ như toán tử ++ hoặc --*
- Các toán tử kết hợp 2 hoặc nhiều giá trị, ví dụ như toán tử +, -, *, /*
- Toán tử ?: dùng để chọn 1 trong 2 giá trị*

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

4. Toán tử :

Toán tử	Sử dụng	Tương tự
+=	<code>\$a+= \$b</code>	<code>\$a*= \$a+\$b</code>
++	<code>\$a++</code>	<code>\$a*= \$a+1</code>
-=	<code>\$a-= \$b</code>	<code>\$a*= \$a-\$b</code>
--	<code>\$a--</code>	<code>\$a*= \$a-1</code>
=	<code>\$a= \$b</code>	<code>\$a*= \$a*\$b</code>
/=	<code>\$a/= \$b</code>	<code>\$a*= \$a/\$b</code>
%=	<code>\$a%= \$b</code>	<code>\$a*= \$a%\$b</code>
.=	<code>\$a.= \$b</code>	<code>\$a*= \$a.\$b</code>

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

5. Các phép toán cơ bản :

a. Các phép toán số học :

$+$, $-$, $*$, $/$, $\%$ (*module*)

b. Các phép toán về chuỗi : Ta sử dụng dấu $.$ để nối 2 chuỗi.

Ví dụ:

```
<?php $a= "Xin chao"; $b="cac ban";  
      $a=$a . "tat ca";  
      echo $a . $b;  
?  
>
```

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

5. Các phép toán cơ bản :

c. Các phép toán so sánh :

Phép toán so sánh dùng để so sánh 2 giá trị, nếu sử dụng để so sánh 2 biểu thức, kết quả trả về là True hay False tùy vào kết quả so sánh.

Toán tử	Sử dụng	Tương tự
==	\$a==\$b	\$a bằng \$b
===	\$a=== \$b	\$a bằng và cùng kiểu \$b
!=	\$a!=\$b	\$a khác \$b
<>	\$a<>\$b	\$a khác \$b
<	\$a<\$b	\$a nhỏ hơn \$b
<=	\$a<=\$b	\$a nhỏ hơn hoặc bằng \$b
>	\$a>\$b	\$a lớn hơn \$b
>=	\$a>=\$b	\$a lớn hơn hoặc bằng \$b

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

5. Các phép toán cơ bản :

d. Phép toán logic

! Phép toán phủ định (not)

&& Phép toán và (and)

|| Phép toán hoặc (or)

and Phép toán và (and)

or Phép toán hoặc (or).

e. Phép toán Error

Trong trường hợp biểu thức hay phép toán báo lỗi, nhưng ta không muốn trình duyệt hiển thị lỗi ta dùng phép toán @ trước biểu thức

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

5. Các phép toán cơ bản :

e. Phép toán Error

Ví dụ:

```
<?php
```

```
$a=10;
```

```
$b=0;
```

```
$c=@($a/$b);
```

```
echo "Gia tri c = ".$c;
```

```
?>
```

NGÔN NGỮ LẬP TRÌNH PHP

III. Lập trình PHP

6. Biến hàm

Là các thư viện hàm trong PHP cho phép thao tác và kiểm tra biến với nhiều cách khác nhau.

Các hàm sau trả về true nếu đúng, ngược lại là sai

- `is_array()` : Kiểm tra biến có phải kiểu mảng không
- `is_double`: Kiểm tra biến có phải kiểu double không
- `is_float()` : Kiểm tra biến có phải kiểu float không
- `is_integer()` : Kiểm tra biến có phải kiểu int không
- `is_string()` : Kiểm tra biến có phải kiểu chuỗi không
- `is_object()` : Kiểm tra biến có phải kiểu đối tượng không

CẤU TRÚC ĐIỀU KHIỂN

I. Cấu trúc lựa chọn :

1. Câu lệnh *if*

Cú pháp đơn giản nhất của câu lệnh *if* có dạng như sau:

***if* (biểu thức) câu lệnh;**

Câu lệnh *if* trên được diễn giải như sau: nếu biểu thức trả về giá trị TRUE (hoặc tương đương với TRUE sau khi chuyển đổi) thì câu lệnh sẽ được thực thi; ngược lại (khi biểu thức trả về giá trị FALSE) thì bỏ qua không thực thi câu lệnh nữa.

Cú pháp nâng cao của câu lệnh if có dạng như sau:

if (biểu thức) câu lệnh 1;
else câu lệnh 2;

Câu lệnh if trên được diễn giải như sau: nếu **biểu thức** trả về giá trị TRUE thì **câu lệnh 1** sẽ được thi hành, ngược lại thì **câu lệnh 2** sẽ được thi hành.

Các lệnh if có thể được lồng vào nhau để tạo ra câu lệnh if phức tạp hơn:

if (biểu thức 1)

if (biểu thức 2) câu lệnh 1;

else câu lệnh 2;

else câu lệnh 3;

Nếu biểu thức 1 trả về giá trị FALSE thì câu lệnh 3 sẽ được thực hiện, ngược lại xét tiếp biểu thức 2: nếu biểu thức 2 trả về giá trị TRUE thì thực hiện câu lệnh 1, ngược lại nếu biểu thức 2 trả về giá trị FALSE thì thực hiện câu lệnh 2.

*if (biểu thức 1) câu lệnh 1;
else if (biểu thức 2) câu lệnh 2;
else if (biểu thức 3) câu lệnh 3;
else câu lệnh 4;*

Nếu **biểu thức 1** trả về TRUE thì thực hiện **câu lệnh 1** (các câu lệnh 2,3,4 không thực hiện), nếu **biểu thức 1** trả về FALSE và **biểu thức 2** trả về TRUE thì **câu lệnh 2** được thực hiện, nếu **biểu thức 1** trả về FALSE **biểu thức 2** trả về FALSE và **biểu thức 3** trả về TRUE thì **câu lệnh 3** được thực hiện. Nếu cả 3 biểu thức 1,2,3 đều trả về FALSE thì thực hiện **câu lệnh 4**.

Ngoài ra PHP còn cung cấp từ khoá **elseif**, chính là ghép giữa từ khoá **else** và **if**.

2. Câu lệnh switch

Cú pháp :

```
switch ($bt) {  
    case h1: các lệnh 1;  
        break;  
    case h2: các lệnh 2;  
        break;  
    .....  
    case hn: các lệnh n;  
        break;  
    default : các lệnh n+1;  
}
```

2. Câu lệnh switch

```
<?php
    switch ($x) {
        case 1: echo "Number 1";
                break;
        case 2: echo "Number 2";
                break;
        case 3: echo "Number 3";
                break;
        default : echo "No number between 1 and 3";
    }
?>
```

2. Câu lệnh switch

Câu lệnh switch hoạt động như là 1 loạt câu lệnh if ghép lại với nhau. Ta hãy xem câu lệnh if sau:

```
if ( $a == "abc" )
```

```
    {echo "Giá trị của a là abc"; }
```

```
else if ( $a == "def" )
```

```
    {echo "Giá trị của a là def"; }
```

```
else if ( $a == "123" )
```

```
    { echo "Giá trị của a là 123"; }
```

```
else
```

```
    { echo "Giá trị khác"; }
```

Có thể được viết lại bằng câu lệnh switch như sau:

```
switch ( $a )
```

```
{ case "abc" : echo "Giá trị của a là abc";  
  break;
```

```
  case "def": echo "Giá trị của a là def";  
  break;
```

```
  case "123": echo "Giá trị của a là 123";  
  break;
```

```
  default: echo "Giá trị khác";
```

```
}
```

```
//end switch
```

Cấu trúc điều khiển

II. Cấu trúc lệnh lặp:

1. Câu lệnh *while* :

while (biểu thức) câu lệnh;

Trong khi biểu thức còn trả về giá trị TRUE thì tiếp tục thực hiện câu lệnh, sau khi thực hiện câu lệnh thì kiểm tra lại biểu thức, nếu vẫn còn trả về giá trị TRUE thì lại tiếp tục thực hiện câu lệnh...cứ tiếp tục như vậy cho tới khi nào biểu thức trả về giá trị FALSE thì ngưng.

Ví dụ: In ra các số từ 1 tới 10 với câu lệnh while:

```
<?php $i = 1;
while ( $i <= 10 )
    { echo $i, "<br>";
      $i++;
    }
//end while
?>
```

Ghi chú: Câu lệnh **\$i++** tương đương với **\$i = \$i+1;**, câu lệnh này sẽ tăng giá trị của \$i lên 1 qua mỗi lần lặp.

2. Câu lệnh *do-while*

Câu lệnh *do-while* cũng tương tự như câu lệnh **while**, chỉ khác một điểm là câu lệnh được thực hiện trước rồi biểu thức mới được kiểm tra sau, nếu biểu thức còn trả về giá trị TRUE thì tiếp tục thực hiện câu lệnh.

Cú pháp của câu lệnh *do-while* như sau:

do

{ câu lệnh;

}

while (biểu thức);

Ví dụ:

In ra các số từ 1 tới 10 với câu lệnh **do-while**:

```
<?php $i = 1;
```

```
do
```

```
{ echo $i, "<br>";
```

```
  $i++; }
```

```
while ( $i <= 10 );
```

```
?>
```

3. Câu lệnh *for*

Câu lệnh *for* cũng dùng để tạo vòng lặp. Đây là một trong những câu lệnh phức tạp nhất của PHP, cú pháp của nó như sau:

*for (biểu thức 1; biểu thức 2; biểu thức 3)
câu lệnh;*

Đầu tiên biểu thức 1 được thực hiện, Tiếp theo biểu thức 2 được kiểm tra Nếu trả về TRUE thì câu lệnh được thực hiện và sau đó thực hiện biểu thức 3. Nếu trả về FALSE thì kết thúc câu lệnh *for*. Kiểm tra lại biểu thức 2 và lặp lại quá trình như trên.

Ví dụ:

In ra các số từ 0 tới 9 với câu lệnh for:

```
<?php for ( $i = 0; $i < 10; $i++ )  
    { echo $i, "<b>"; }  
    //end for  
?>
```

4. Câu lệnh *foreach*

Câu lệnh *foreach* chỉ làm việc với array. Câu lệnh *foreach* có 2 dạng cú pháp như sau:

foreach (\$array as \$value) câu lệnh;

foreach (\$array as \$key => \$value) câu lệnh;

Ví dụ 1:

```
<?php
```

```
$a = array('a' => '1', 'b' => '2', 'c' => '3');
```

```
foreach ( $a as $value )
```

```
    { echo $value, "\n"; }
```

```
//end foreach
```

```
?>
```

Chương trình trên sẽ in ra 3 số 1, 2 và 3.

Ví dụ 2:

```
<?php
    $a = array('a' => '1', 'b' => '2', 'c' => '3');
    foreach ( $a as $key => $value )
        { echo $key, "=", $value, "\n"; }
    //end foreach
?>
```

Chương trình trên sẽ in ra 3 chuỗi **a=1**, **b=2** và **c=3**.

5. Câu lệnh *break*

Câu lệnh *break* sẽ dừng việc thực thi của các vòng lặp *for*, *foreach*, *while*, *do-while* và *switch*.

Ở phần trước ta đã thấy câu lệnh *break* được sử dụng trong câu lệnh *switch*.

Ví dụ: sử dụng câu lệnh *break* trong vòng lặp *for*:

```
for ( $i=1; $i<=10; $i++ )  
    { echo $i;  
      if ( $i == 5 ) break;  
    }
```

6. Câu lệnh continue

Câu lệnh continue áp dụng lên các vòng lặp, lệnh continue sẽ bỏ qua lần lặp hiện thời và tiếp tục thực hiện các lần lặp tiếp theo.

Ví dụ:

```
for ( $i=1; $i<=5; $i++ )  
    { if ( $i == 2 ) continue;  
      echo $i; }
```

Khi \$i đạt giá trị 2, câu lệnh **echo \$i;** sẽ được bỏ qua không thì hành nữa do câu lệnh continue. Và như vậy, đoạn lệnh trên khi chạy sẽ in ra các giá trị 1,3,4,5 (không có giá trị 2).

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU **MYSQL**

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

1. Cơ sở dữ liệu MySQL:

- + Để có thể liên kết và truy xuất dữ liệu trong PHP, trước tiên ta cần tìm hiểu cách thao tác làm việc trên cơ sở dữ liệu trong MySQL.
- + MySQL là một hệ quản trị cơ sở dữ liệu vừa và nhỏ.
- + Khi làm việc với cơ sở dữ liệu MySQL, bạn *đăng ký kết nối, tạo cơ sở dữ liệu, quản lý người dùng, phân quyền sử dụng, thiết kế đối tượng Table của cơ sở dữ liệu và xử lý dữ liệu.*

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

1. Cơ sở dữ liệu MySQL:

+ Nếu lần đầu tiên tạo kết nối cơ sở dữ liệu, chúng ta cần phải tạo một Connection, cung cấp tên Server hay IP của máy chứa MySQL.

+ Trong trường hợp máy chứa cơ sở dữ liệu MySQL là máy đang sử dụng, bạn có thể sử dụng tên `localhost`. Username mặc định của cơ sở dữ liệu MySQL là `root` và Password là `root`.

+ Cuối cùng bạn bắt đầu tạo cơ sở dữ liệu và làm việc với cơ sở dữ liệu.

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

a. Thiết lập kết nối cơ sở dữ liệu: PHP cung cấp cho chúng ta hàm kết nối cơ sở dữ liệu có cú pháp:

int mysql_pconnect("servername", "username", "password")

Ngoài ra có thể sử dụng phát biểu như sau:

\$link=mysql_connect("servername", "username", "password") or die("Could not connect to MySQL Database");

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

b. Mở một cơ sở dữ liệu :

+ Sau khi khai báo kết nối đến cơ sở dữ liệu MySQL thành công, bạn khai báo phát biểu để mở 1 cơ sở dữ liệu.

+ Nếu chọn cách 1 thì ta khai báo:

- *mysql_select_db("tên CSDL");*

+ Nếu chọn cách 2 thì ta khai báo:

- *mysql_select_db("tên CSDL",\$link);*

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

Ví dụ: Trong CSDL MySQL có cơ sở dữ liệu có tên là **DULIEU**. Để thực hiện kết nối đến cơ sở dữ liệu này ta khai báo đoạn lệnh sau:

```
<?php
```

```
//Khai báo kết nối
```

```
$link = mysql_connect("localhost","root","") or die ("Khong the  
ket noi den CSDL MySQL");
```

```
//Lựa chọn cơ sở dữ liệu
```

```
mysql_select_db("DULIEU", $link);
```

```
?>
```

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

c. Đóng kết nối dữ liệu :

+ Khi không sử dụng kết nối cơ sở dữ liệu, bạn có thể khai báo phát biểu đóng kết nối như sau:

`mysql_close(database_connection);`

Ví dụ: Đóng kết nối cơ sở dữ liệu trên:

`mysql_close($link);`

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

d. Truy vấn cơ sở dữ liệu:

+ Để truy vấn cơ sở dữ liệu ta có thể dùng hàm `mysql_query()` với tham số bắt buộc là chuỗi SQL, chuỗi kết nối là tham số tùy chọn như một trong 3 phát biểu sau:

```
int mysql_query( string sql);
```

```
int mysql_query( string sql, [int db_connection]);
```

```
int mysql_query( string database, string sql, [int  
db_connection]);
```

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

```
2 //Khai báo ket noi
3 $link = mysql_connect("localhost","root","") or die ("Khong the ket noi den CSDL MySQL");
4 //Lua chon co so du lieu
5 // mysql_select_db("DULIEU",$link);
6 mysqli_select_db($link,"DULIEU");
7 $sql="select * from Table1";
8 //$result=mysql_query($sql,$link);
9 $result=mysqli_query($link,$sql);
```

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU MYSQL

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

+ Để biết được số lượng mẫu tin trả về ta có thể dùng hàm :

int mysql_num_rows();

+ Bạn có thể đọc từng mẫu tin bằng các hàm sau:

mysql_fetch_row();

mysql_fetch_assoc();

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

```
<?php
```

```
$link = mysql_connect("localhost", "user", password");  
mysql_select_db("database", $link);
```

```
$result = mysql_query("SELECT * FROM table1", $link);  
$num_rows = mysql_num_rows($result);
```

```
echo "$num_rows Rows\n";
```

```
?>
```

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

```
$row = mysql_fetch_row($result);
```

```
$row = mysql_fetch_assoc($result);
```

Kết quả trả về từ 2 hàm này là một **array**. Hai hàm này trả về dòng kết quả hiện thời của câu lệnh select và chuyển con trỏ tới dòng tiếp theo hoặc giá trị FALSE nếu như không còn dòng nào để trả về nữa.

```
mysql_error($conn);
```

Trả về thông báo lỗi của MySQL server nếu như một lệnh trước đó có lỗi.

*Sự khác nhau của **mysql_fetch_row()** và **mysql_fetch_assoc()***

+ Hàm **mysql_fetch_row()** sẽ trả về một array mà phần tử thứ [0] sẽ tương ứng với cột đầu tiên của table, phần tử thứ [1] sẽ tương ứng với cột thứ hai của table, ..

Ví dụ : `while ($row = mysql_fetch_row($result))
 { echo "Username = ".$row[0]."
\n";
 echo "Password = ".$row[1]."
\n"; }`

+ Hàm **mysql_fetch_assoc()** sẽ trả về một array mà các phần tử sẽ được truy cập qua tên, với tên được lấy từ tên các cột của table

Ví dụ : `while ($row = mysql_fetch_assoc($result))
 { echo "Username = ".$row['username']. "
\n";
 echo "Password = ".$row['password']. "
\n"; }`

I. Truy xuất cơ sở dữ liệu bằng PHP

2. Truy xuất cơ sở dữ liệu trong PHP:

```
1 <?php
2 //Khai báo kết nối
3 $link = mysql_connect("localhost","root","") or die ("Không thể kết nối đến CSDL MySQL");
4     //Lựa chọn cơ sở dữ liệu
5     // mysql_select_db("DULIEU",$link);
6     mysqli_select_db($link,"DULIEU");
7     $sql="select * from Table1";
8     //$result=mysql_query($sql,$link);
9     $result=mysqli_query($link,$sql);
10 echo '<table border="1" width="100%">';
11 echo '<caption>Dữ liệu truy xuất từ bảng Nhân sự</caption>';
12 //in Tiêu đề của bảng
13 echo '<tr><th>Mã Số</th><th>Họ tên</th><th>Ngày sinh</th><th>Nghề nghiệp</th></tr>';
14 while($row=mysqli_fetch_array($result)){
15 // while($row=mysql_fetch_array($result))
16     $maso=$row["maso"];
17     $hoten=$row["hoten"];
18     $ngaysinh=$row["ngaysinh"];
19     $nghenghiep=$row["nghenghiep"];
20     echo "<tr> <td>$maso</td><td>$hoten</td><td>$ngaysinh</td><td>$nghenghiep</td></tr>";
21 }
22 echo '</table>';
23 //Giải phóng tập các bản ghi trong $rs
24 mysql_free_result($result);
25 mysql_close($link);
26 ?>
```

THỰC THI CÂU LỆNH SELECT

```
<?php
$result = mysql_query("SELECT id,email
FROM people WHERE id = '42' ");
if (!$result) {
    echo 'Could not run query: ' . mysql_error();
    exit;
}
$row = mysql_fetch_row($result);

echo $row[0]; // 42
echo $row[1]; // the email value
?>
```

.....

.....

```
while ($row = mysql_fetch_assoc($result)) {  
    echo $row["userid"];  
    echo $row["fullname"];  
    echo $row["userstatus"];  
}
```

```
mysql_free_result($result);
```

```
?>
```

THỰC THI CÂU LỆNH INSERT

```
<?php
$link = mysql_connect('localhost', 'user', 'password');
    if (!$link)
    {
        die('Could not connect: ' . mysql_error());
    }
    mysql_select_db('mydb', $link);
    $sql = "INSERT INTO thanhvien (id, user, pass, email) VALUES (1,
        'admin', '123')";
    $result = mysql_query($sql, $link);
        //chèn thêm 1 dòng vào table
    if ( !$result ) die("Không thể thực hiện được câu lệnh
        SQL:".mysql_error($link));
    echo "Số lượng row được chèn: ".mysql_affected_rows($link)."<br>\n";
    //ta chỉ chèn 1 dòng nên hàm mysql_affected_rows sẽ trả về 1
?>
```

THỰC THI CÂU LỆNH UPDATE

```
<?php
$link = mysql_connect('localhost', 'user', 'password');
    if (!$link)
    {
        die('Could not connect: ' . mysql_error());
    }
mysql_select_db('mydb', $link);
$sql = "UPDATE thanhvien SET pass='abc' WHERE username='admin'";
$result = mysql_query($sql, $link);
    //đổi password của admin
if ( !$result ) die("Không thể thực hiện được câu lệnh SQL:
    ".mysql_error($link));
echo "Số lượng row được thay đổi:
    ".mysql_affected_rows($link)."<br>\n";
?>
```

THỰC THI CÂU LỆNH DELETE

```
<?php
$link = mysql_connect('localhost', 'user', 'password');
    if (!$link)
    {
        die('Could not connect: ' . mysql_error());
    }

mysql_select_db('mydb', $link);
$sql = "DELETE * FROM thanhvien";
$result = mysql_query($sql, $link);
//xóa hết tất cả các account
if ( !$result ) die("Không thể thực hiện được câu lệnh SQL:
    ".mysql_error($link));
echo "Số lượng row được xóa: ".mysql_affected_rows($conn)."<br>\n";
?>
```

HÀM

1. Định nghĩa :

Trong lập trình, có một số đoạn mã được dùng nhiều lần ở nhiều nơi khác nhau trong chương trình. PHP cung cấp một giải pháp đó là hàm do người dùng định nghĩa.

Ta có thể đưa đoạn mã đó vào trong một hàm, và ở chỗ nào cần dùng đoạn mã đó ta chỉ cần gọi hàm, khi cần sửa đổi, ta chỉ cần sửa đổi một chỗ duy nhất là nội dung của hàm chứ không cần phải sửa ở nhiều nơi trong chương trình.

Cú pháp để tạo hàm như sau:

```
function tênHàm($tham_số1, $tham_số2, ...,)  
{  
    // các lệnh trong thân hàm ;  
    return $kết_quả_trả_về;  
}  
//end
```

Khi cần sử dụng hàm ở chỗ nào, ta chỉ cần dùng cú pháp :

tênHàm(các tham số)

Ví dụ:

```
<?php
```

```
function testing()
```

```
{ echo "Testing function";
```

```
echo 1; echo 2; echo 3;
```

```
} //end testing
```

```
testing(); testing(); testing();
```

```
?>
```

Tên hàm cũng như tên biến , nhưng được phép bắt đầu bằng ký tự gạch dưới .

Tham số của hàm

Hàm có thể nhận vào các tham số, ví dụ:

```
<?php
```

```
function testing($a)
```

```
{ echo "Tham số là $a"; }
```

```
//end testing testing(123);
```

```
testing("abc");
```

```
?>
```

Ở ví dụ trên, lời gọi hàm `testing(123);` sẽ in ra dòng Tham số là 123 và lời gọi hàm `testing("abc");` sẽ in ra dòng Tham số là abc.

Ta có thể gán giá trị mặc định cho tham số của hàm:

```
<?php
    function testing($a="mặc định")
        { echo "Tham số là $a";
        } //end

testing();
?>
```

Khi tham số tương ứng của hàm không được truyền, tham số đó sẽ nhận giá trị mặc định. Đoạn chương trình ví dụ ở trên khi chạy sẽ in ra dòng Tham số là mặc định.

Giá trị trả về từ hàm

Hàm còn thể trả về 1 giá trị cho nơi gọi:

```
<?php
```

```
function binh_phuong($a)
```

```
{    $ketqua = $a * $a;
```

```
    return $ketqua;
```

```
} //end testing
```

```
echo binh_phuong(2);
```

```
?>
```

Đoạn chương trình trên khi chạy sẽ in ra số 4. Câu lệnh `return biểu_thức;` sẽ kết thúc hàm và trả về giá trị của `biểu_thức` cho nơi gọi.

Chú ý: bên trong hàm, biến cục bộ sẽ được dùng thay vì biến toàn cục.

Ví dụ:

```
<?php
```

```
$a = 1; //biến toàn cục //hàm do tự tạo
```

```
function test()
```

```
    { echo $a; }
```

```
//end test
```

```
?>
```

Để truy cập tới các biến toàn cục ở bên trong một hàm do người dùng định nghĩa, ta có thể dùng một trong hai cách sau:

Cách 1:

```
<?php $a = 1; //biến toàn cục  
//hàm do tự tạo  
function test()  
{ //từ khoá global báo cho php biết là bên test  
  //bây giờ ta sẽ dùng biến toàn cục  
  global $a;  
  echo $a; //in ra giá trị: 1  
} //end test
```

Cách 2:

```
<?php
```

```
$a = 1; //biến toàn cục
```

```
//hàm do tự tạo
```

```
function test()
```

```
{ echo $GLOBALS['a']; //in ra giá trị: 1
```

```
} //end test
```

```
?>
```

2. Các hàm thông dụng của PHP

a. Hàm xem thông tin **phpinfo()**

Các tham số:

INFO_GENERAL

INFO_CREDITS

INFO_CONFIGURATION

INFO_MODULES

INFO_ENVIRONMENT

INFO_VARIABLES

INFO_LICENSE

INFO_ALL

```
<html>
<body>
  <?php
      phpinfo();
  ?>
  <?php
      phpinfo(INFO_GENERAL);
  ?>
</body>
</html>
```

b. Hàm hiển thị thông tin Server

```
<?php
    echo "Hỗ trợ: " .
    $_SERVER["HTTP_REFERER"] . "<br />";
    echo "Hỗ trợ Browser: " .
    $_SERVER["HTTP_USER_AGENT"] . "<br />";
    echo "Địa chỉ IP: " .
    $_SERVER["REMOTE_ADDR"];
?>
```

c. Hàm Header()

Chuyển đến một trang nào đó

```
<?php
```

```
    header("Location: http://dut.udn.vn/");
```

```
?>
```

```
<html> <body>
```

```
.....</body> </html>
```

Lưu ý: Khi dùng hàm này cần:

- * Phải đảm bảo các lệnh trước đó không có tác dụng xuất bất cứ dữ liệu nào ra trình duyệt thì nó mới có tác dụng (*kể cả trước đó là các thẻ HTML*)

3. Truy cập biến từ form

Trong PHP khai báo để lấy giá trị biến form từ trang HTML tương đối đơn giản, tên của một đối tượng Form trong HTML cũng chính là tên biến dùng trong PHP.

VD:

```
<?php echo "ten cua ban la: ".$txtUser;  
        echo "Mat khau cua ban la: ".$txtPass;  
?>
```

Trong đó txtUser và txtPass là tên đối tượng dạng text trong thẻ form.

Sử dụng mảng hằng

```
$HTTP_POST_VARS["tên biến form"];  
$HTTP_GET_VARS["tên biến form"];
```

VD:<?php

```
echo "ten cua ban la: ".  
$HTTP_POST_VARS ["txtUser"];  
?>
```

Trong đó txtUser là tên đối tượng dạng text trong thẻ form.

4. Khai báo hằng trong PHP

- + Tất cả các hằng số trong PHP là chữ hoa

- + Cú pháp khai báo:

`define("TENHANG", giá trị gán hằng)`

Ví dụ:

```
define("PI",3.14);
```

5. Sử dụng lại mã chương trình trong PHP

PHP cung cấp hai cách để khai báo sử dụng bất kỳ đoạn mã chương trình nào đã có sẵn, bằng cách sử dụng hai khai báo

require()** và **include()

a. Sử dụng khai báo ***require()*** :

Cú pháp: ***require(string path_file);***

Ví dụ : Tạo 1 file require1.php có nội dung như sau:

```
<?php
$a=2;
$b=3;
$c=$a+$b;
echo "a + b = ".$c."<br>";
?>
```

- tạo một file php có nội dung

<?php

require("require1.php");

?>

Các lưu ý:

- File được gọi có phần mở rộng tùy ý.
- Các đoạn mã PHP trong File được gọi phải đặt trong cặp thẻ *<?php ...?>*

b. Sử dụng khai báo *include()* :

Cú pháp: *include(string path_file);*

Giống như trường hợp trên, nhưng *include()* và *require()* khác nhau là: Tất cả nội dung được nạp lên bằng *require()* *webserver* phải dịch tất cả nội dung bên trong.

Do đó trong những file nạp bằng *require()* không nên dùng các cấu trúc điều khiển.

c. Khác nhau giữa `require` và *`include()`* :

- + `require()` sẽ tạo ra cảnh báo lỗi và dừng chương trình ngay lập tức.
- + `include()` sẽ tạo ra cảnh báo nhưng chương trình vẫn tiếp tục chạy.

c. Khác nhau giữa require và *include()* :

Ví dụ: Đoạn code trong test.php

- <?php
require “import.php”;
echo “This is a file: test.php”;
?>

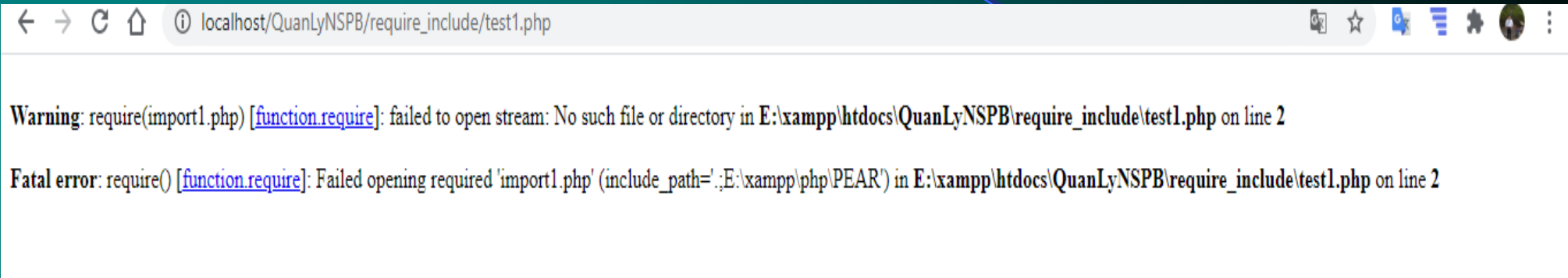
Đoạn code trong import.php

- <?php
echo “This is a file: import.php
”;
?>

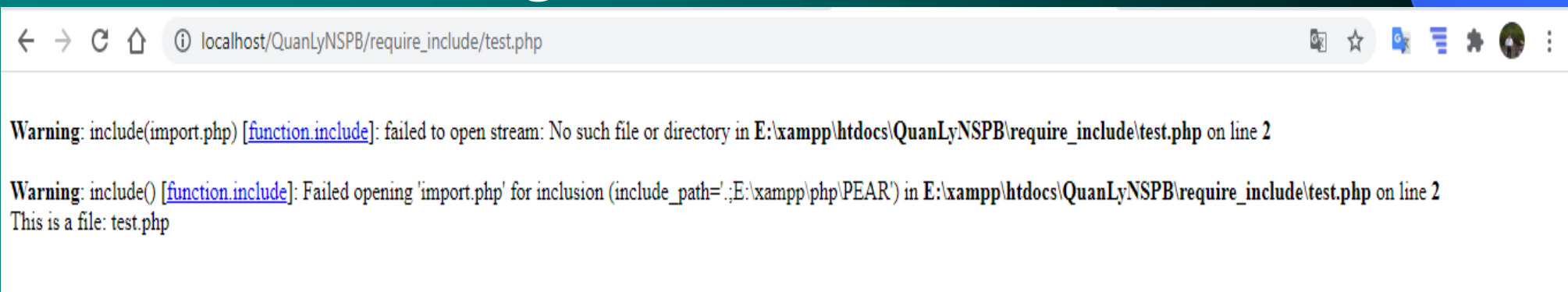
Kết quả: **This is a file: import.php**
This is a file: test.php

c. Khác nhau giữa `require` và `include()` :

+ Nhưng nếu bây giờ **xoá file `import.php`** đi thì chương trình sẽ không chạy nữa.



+ Cũng ví dụ trên ta thay **`require`** bằng **`include`**. Ta thấy file **`test.php`** vẫn thực thi bình thường nhưng có xuất hiện thông báo lỗi.



d. Sử dụng `require_once` và *`include_once`* :

+ Khi viết code PHP nếu sử dụng `include()` hoặc `require()` hơn 1 lần sẽ rất dễ xảy ra xung đột. Vì thế lúc này cần dùng tới `require_once()`

Ví dụ: Đoạn code trong `test2.php`

```
<?php
require_once "import.php";
require_once "import.php";
echo "This is a file: test2.php";
?>
```

+ Mặc dù `require_once()` được thêm 2 lần nhưng nó chỉ chạy 1 lần duy nhất mà thôi. Còn file 2 sẽ không có tác dụng gì cả. Chương trình thông báo lỗi và dừng ngay tương tự như `require()`.

d. Sử dụng `require_once` và `include_once` :

+ `include_once` tương tự như `include()`, nó sẽ import tập tin chỉ 1 lần duy nhất tuy xuất hiện cảnh báo nhưng chương trình vẫn hoạt động.

Ví dụ: Đoạn code trong `test3.php`

- ```
<?php
include_once "import.php";
include_once "import.php";
echo "This is a file: test3.php";
?>
```

+ Mặc dù `include_once` được thêm 2 lần nhưng nó chỉ chạy 1 lần duy nhất mà thôi. Còn file 2 sẽ không có tác dụng gì cả. Chương trình đưa ra thông báo lỗi nhưng vẫn hoạt động bình thường.

# CÁC HÀM XỬ LÝ CHUỖI TRONG PHP

Xử lý chuỗi trong ứng dụng web là một phần chính của PHP, điểm mạnh của PHP là khả năng xử lý chuỗi.

## 1. Hàm xử lý khoảng trắng trên chuỗi:

+Hàm kiểm tra chuỗi rỗng:

Cú pháp: *isempty(string str);*

Nếu chuỗi str rỗng thì trả về true, ngược lại trả về false.

+Hàm xóa bỏ khoảng trắng bên trái và bên phải chuỗi

Cú pháp: *ltrim(string str);*

và: *chop(string str);*

Ví dụ:

```
$name=ltrim($name);
```

```
$pass=chop($pass);
```

## 2. Hàm thay đổi kiểu chữ của chuỗi:

| Hàm             | Diễn giải                | Sử dụng                      | Kết quả    |
|-----------------|--------------------------|------------------------------|------------|
| <b>strupper</b> | Chữ hoa                  | <b>strupper("toi")</b>       | TOI        |
| <b>strlower</b> | Chữ thường               | <b>strlower ("TOI")</b>      | toi        |
| <b>ucfirst</b>  | Chữ hoa ký tự đầu tiên   | <b>ucfirst("toi va ban")</b> | Toi va ban |
| <b>ucword</b>   | Chữ hoa ký tự đầu mỗi từ | <b>ucword("toi va ban")</b>  | Toi Va Ban |

### 3. Định dạng chuỗi để đưa vào và lấy ra từ CSDL:

Để lưu trữ một chuỗi vào CSDL mà chuỗi đó cho chứa các ký tự ' hay " hay \ thì phải thêm dấu \ vào trước khi lưu.

Ta sử dụng hàm *AddSlashes(string str)*

Hàm này sẽ tự tìm các ký tự trên và thêm \ vào trước.

Ngược lại ta dùng hàm :

*StripSlashes(string str)* để loại bỏ \.

Ví dụ:

```
$noidung=AddSlashes($noidung);
```

```
$noidung=StripSlashes($noidung);
```

#### 4. Hàm kết hợp hay tách chuỗi:

+Tách chuỗi thành 2 chuỗi con bằng một ký tự phân cách và đưa vào mảng.

Cú pháp:

***array=explode(string phan\_cach, string str);***

Ví dụ : tách địa chỉ Email thành 2 phần:

***\$manga=explode("@",\$email);***

***echo manga[1];***

+ Hàm substr();

Cú pháp:

***string substr(string str, int i [,int l]);***

Hàm này trả về chuỗi con của chuỗi **str**, bắt đầu từ vị trí **i** với **l** ký tự. Tham số **l** là tùy chọn, nếu không có thì trả về đến hết chuỗi **str**

## 5. Hàm so sánh chuỗi:

Ngoài toán tử `==` dùng so sánh 2 chuỗi, chúng ta còn có thể dùng các hàm :

*`int strcmp(string str1, string str2);`*

Nếu `str1==str2` : `=0`

`str1>str2` : `>0`

`str1<str2` : `<0`

So sánh `str1` và `str2` có phân biệt chữ hoa, chữ thường, ngoài ra còn có các hàm

*`int strcasecmp(string str1, string str2);`*

*`int strnatcmp(string str1, string str2);`*

## 6. Hàm kiểm tra chiều dài chuỗi:

cú pháp:

*int strlen(string str);*

## 7. Hàm tìm kiếm và thay thế chuỗi

+ Hàm tìm chuỗi:

*string strstr(string str1, string str2);*

Hàm này tìm chuỗi str2 trong str1, nếu có trả về chuỗi mới là chuỗi bắt đầu bằng chuỗi str2 đến hết chuỗi str1.

Ví dụ:

```
<?php
```

```
$a="xin chao cac ban than men";
```

```
echo strstr($a,"c")."
";
```

```
?>
```

+ Hàm tìm vị trí chuỗi con:

*int strpos(string str1, string str2);*

Hàm này trả về vị trí tìm thấy của chuỗi str2 trong str1, nếu không tìm thấy thì trả về -1, nếu có nhiều chuỗi giống nhau, thì chỉ trả về vị trí tìm thấy đầu tiên (*vị trí của chuỗi tính từ 0*)

Ví dụ:

```
<?php
```

```
$a="xin chao cac ban than men";
```

```
$vitri= strpos($a,"chao");
```

```
echo "vi tri la: ".$vitri."
";
```

```
?>
```

## + Hàm thay thế chuỗi:

*string str\_replace(string str1, string str2, string str);*

. Hàm này trả về 1 chuỗi bằng cách thay thế tất cả các chuỗi str1 trong str bằng chuỗi str2.

. Để không chế chỉ cho thay đổi vị trí ta dùng hàm:

*string substr\_replace(string str1, string str2, int i [,int l]);*

Thay thế str2 vào trong str1, bắt đầu từ ký tự có vị trí là i và có chiều dài là l ký tự

Ví dụ:

```
<?php
```

```
$a="xin chao cac ban va tat ca cac ban than men";
```

```
$vitri= strpos($a,"chao");
```

```
$chuoi1= str_replace("ban","em",$a);
```

```
$chuoi2=substr_replace($chuoi1,"ban",13,3);
```

```
echo "chuoi 1 : ".$chuoi1."
";
```

```
echo "chuoi 2 ".$chuoi2."
";
```

```
?>
```

# CÔNG NGHỆ AJAX

## NỘI DUNG

- 1. Giới thiệu*
- 2. Cơ chế và nguyên lý hoạt động*
- 3. Mô hình tương tác Asynchronous*
- 4. Đối tượng XMLHttpRequest*
- 5. Các công nghệ hỗ trợ liên quan*
- 6. Ajax và ứng dụng*

# CÔNG NGHỆ AJAX

## NỘI DUNG

- + Tài liệu về Ajax từ cuốn sách *Ajax in Action* của Dave Crane và những bài viết về Ajax đã có.
- + Kiến thức về công nghệ Ajax : giới thiệu, cơ chế và mô hình hoạt động của Ajax.
- + Các công cụ liên quan : giới thiệu, tìm hiểu chức năng, vai trò trong công nghệ này.

# CÔNG NGHỆ AJAX

## *I. Khái quát chung :*

### *1. Giới thiệu :*

+ Ajax - Asynchronous JavaScript and XML (JavaScript và XML không đồng bộ) là công cụ cho phép tăng tốc độ ứng dụng Web bằng cách cắt nhỏ dữ liệu và chỉ hiển thị những gì cần thiết thay vì phải tải đi tải lại toàn bộ trang Web.

+ Ajax không phải là một công nghệ đơn lẻ mà nó là tập hợp của nhiều công nghệ.

**Asynchronous Javascript + CSS + DOM + XMLHttpRequest**

# CÔNG NGHỆ AJAX

## *I. Khái quát chung :*

### *1. Giới thiệu :*

+ Ajax bao gồm:

- Thể hiện Web theo tiêu chuẩn XHTML và CSS.
- Nâng cao tính năng động và phản hồi bằng DOM (Document Object Model).
- Trao đổi và truy cập/tác động lên thông tin, dữ liệu bằng XSL và XSLT.
- Nhận thông tin không đồng bộ (Synchronous) dùng XMLHttpRequest.
- Và tất cả các kỹ thuật trên được liên kết lại với nhau bằng Javascript.

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 2. Cơ chế và nguyên lý hoạt động :

\* **Cơ chế** : Như vậy Ajax nó đóng vai trò như một lớp trung gian giữa giao diện trên trình duyệt và máy chủ xử lý thông tin. Có thể mô tả một cách cụ thể cách thức hoạt động của Ajax như sau :

- Ajax thực hiện tương tác với máy chủ bằng cách sử dụng đối tượng XMLHttpRequest, nhận kết quả trả về dưới dạng XML và phân tích kết quả bằng công nghệ DOM

- Tương tác giữa Ajax và giao diện người dùng được thực hiện thông qua các mã Javascript và XHTML+CSS.

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 2. Cơ chế và nguyên lý hoạt động :

\* **Cơ chế** : - Công nghệ Ajax thì sẽ cho phép tạo ra một Ajax Engine. Khi đó các yêu cầu gửi Request và nhận Response sẽ do Ajax Engine thực hiện.

- Web server gửi trả dữ liệu dưới dạng XML và Ajax Engine sẽ tiếp nhận, sau đó thực hiện phân tích chuyển hoá thành XHTML+CSS cho trình duyệt hiển thị.

- Các việc này được thực hiện trên Client nên giảm tải rất nhiều cho Server, đồng thời với người sử dụng thì sẽ thấy kết quả hiển thị ngay tức thì mà không cần nạp lại nguyên cả trang.

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 2. Cơ chế và nguyên lý hoạt động :

*\*Nguyên lý hoạt động : Để minh họa cho cơ chế Ajax xin đưa ra hai mô hình tương tác như sau :*

*+ Mô hình thứ nhất là mô hình tương tác giữa máy chủ và máy khách trong các ứng dụng Web truyền thống .*

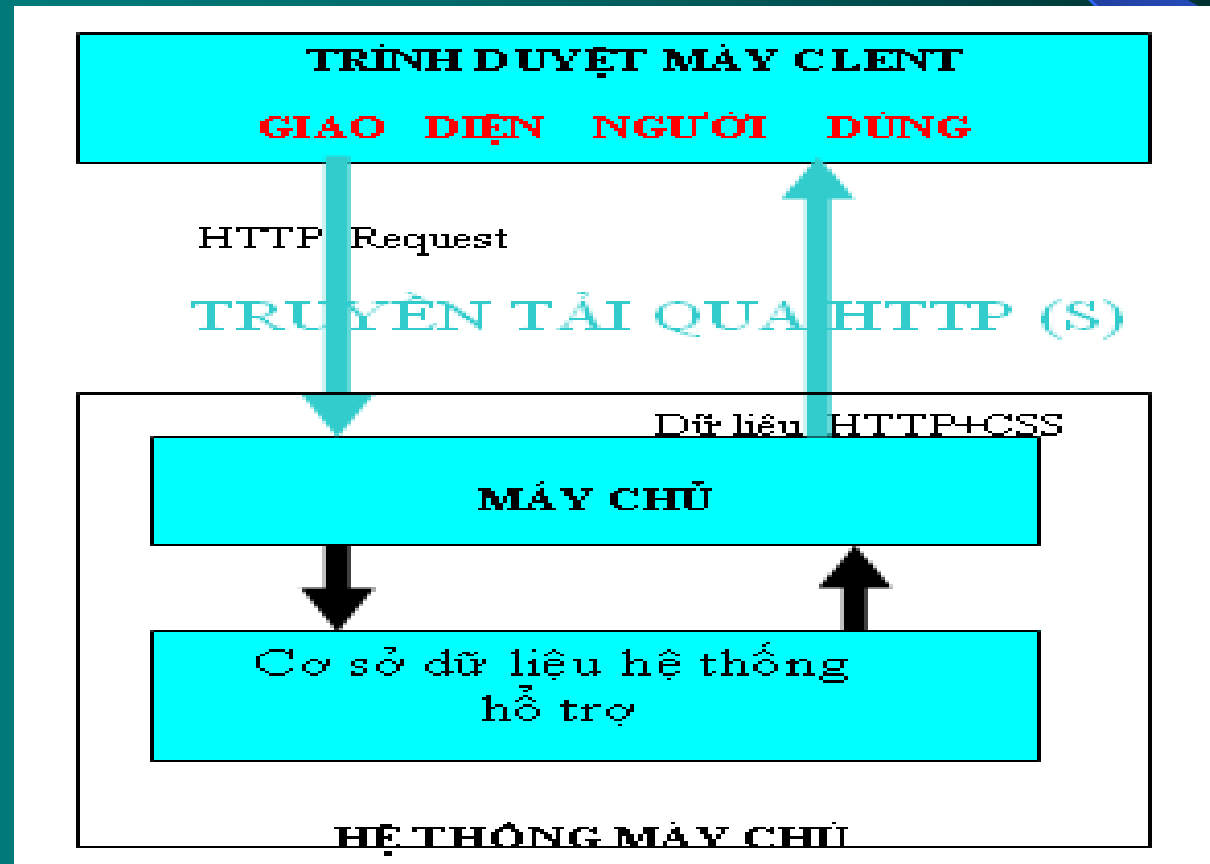
*+ Mô hình thứ hai thể biểu diễn quá trình tương tác giữa máy chủ và máy khách trong các ứng dụng Web có sử dụng công nghệ Ajax.*

# CÔNG NGHỆ AJAX

*I. Khái quát chung :*

*2. Cơ chế và nguyên lý hoạt động :*

*\*Nguyên lý hoạt động :*

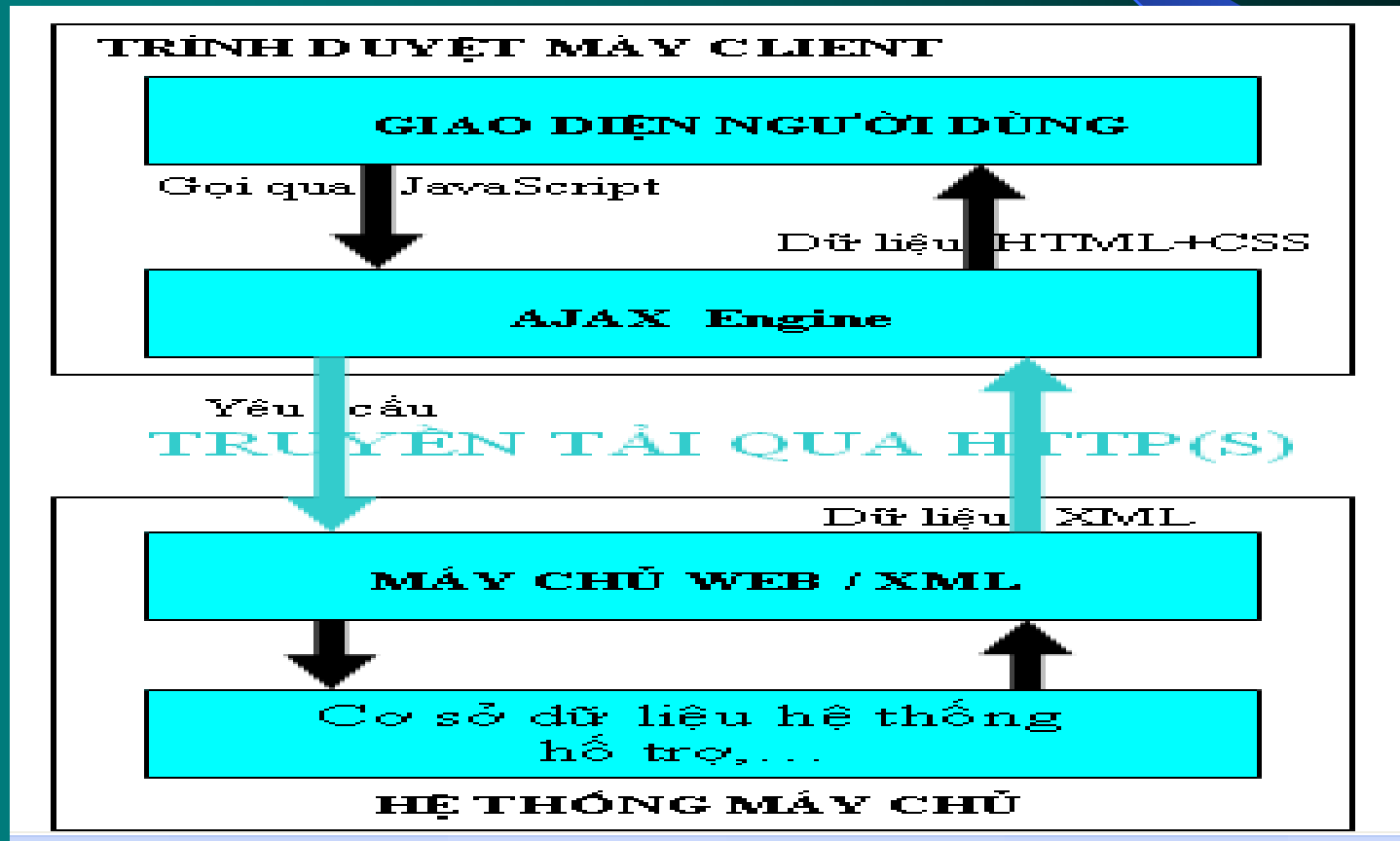


# CÔNG NGHỆ AJAX

*I. Khái quát chung :*

*2. Cơ chế và nguyên lý hoạt động :*

*\*Nguyên lý hoạt động :*



# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 2. Cơ chế và nguyên lý hoạt động :

#### *\*Nguyên lý hoạt động :*

- + *Một trong những điểm mấu chốt của công nghệ Ajax là không tương tác trực tiếp với máy chủ như truyền thống mà là qua một lớp trung gian của Ajax.*
- + *Chính điều đó sẽ tạo ra sự ngạc nhiên về cơ chế xử lý này vì sẽ phải qua thêm một lớp trung gian và như vậy thì sẽ làm chậm đi quá trình tương tác.*

*Tuy nhiên thực tế thì không như vậy, nguyên nhân là :*

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 2. Cơ chế và nguyên lý hoạt động :

#### *\*Nguyên lý hoạt động :*

- Không phải lúc nào cũng cần phải tương tác với máy chủ như trong trường hợp của Gmail, một khi giao diện của Gmail đã được tải về xong thì những tác vụ như chuyển từ thư mục này hay sang xem thư mục khác không nhất thiết phải đòi hỏi gọi lại máy chủ.

- Bằng cách chỉ nhận những thông tin cần thiết (ở dạng XML), dung lượng truyền tải giữa máy khách và máy chủ sẽ giảm đi rất nhiều. Điều này nghĩa là giao diện của ứng dụng chỉ cần tải về một lần (lần đầu)

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 2. Cơ chế và nguyên lý hoạt động :

#### *\*Nguyên lý hoạt động :*

*- Ajax sẽ không phải thực hiện tải lại toàn bộ giao diện đó mỗi khi có tương tác với máy chủ. Thay vào đó Ajax sẽ gửi yêu cầu đến máy chủ và nhận kết quả từ máy chủ về những gì đã thay đổi sau khi máy chủ thực hiện yêu cầu đó.*

*- Ajax sau đó sẽ thông báo cho phần giao diện (thông qua các lời gọi Javascript) thực hiện các thay đổi tương ứng trên giao diện*

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 2. Cơ chế và nguyên lý hoạt động :

#### *\*Nguyên lý hoạt động :*

- Các tương tác giữa phần giao diện với Ajax là tương tác nội bộ trên trình duyệt. Việc này giúp cho việc thực hiện các thay đổi được diễn ra rất nhanh (gần như ngay tức thì).

- Vì thế khi duyệt một trang hỗ trợ Ajax, người sử dụng không bao giờ thấy một cửa sổ trắng (blank) và biểu tượng đồng hồ cát –là những dấu hiệu cho thấy máy chủ đang thực hiện công việc.

# CÔNG NGHỆ AJAX

## *I. Khái quát chung :*

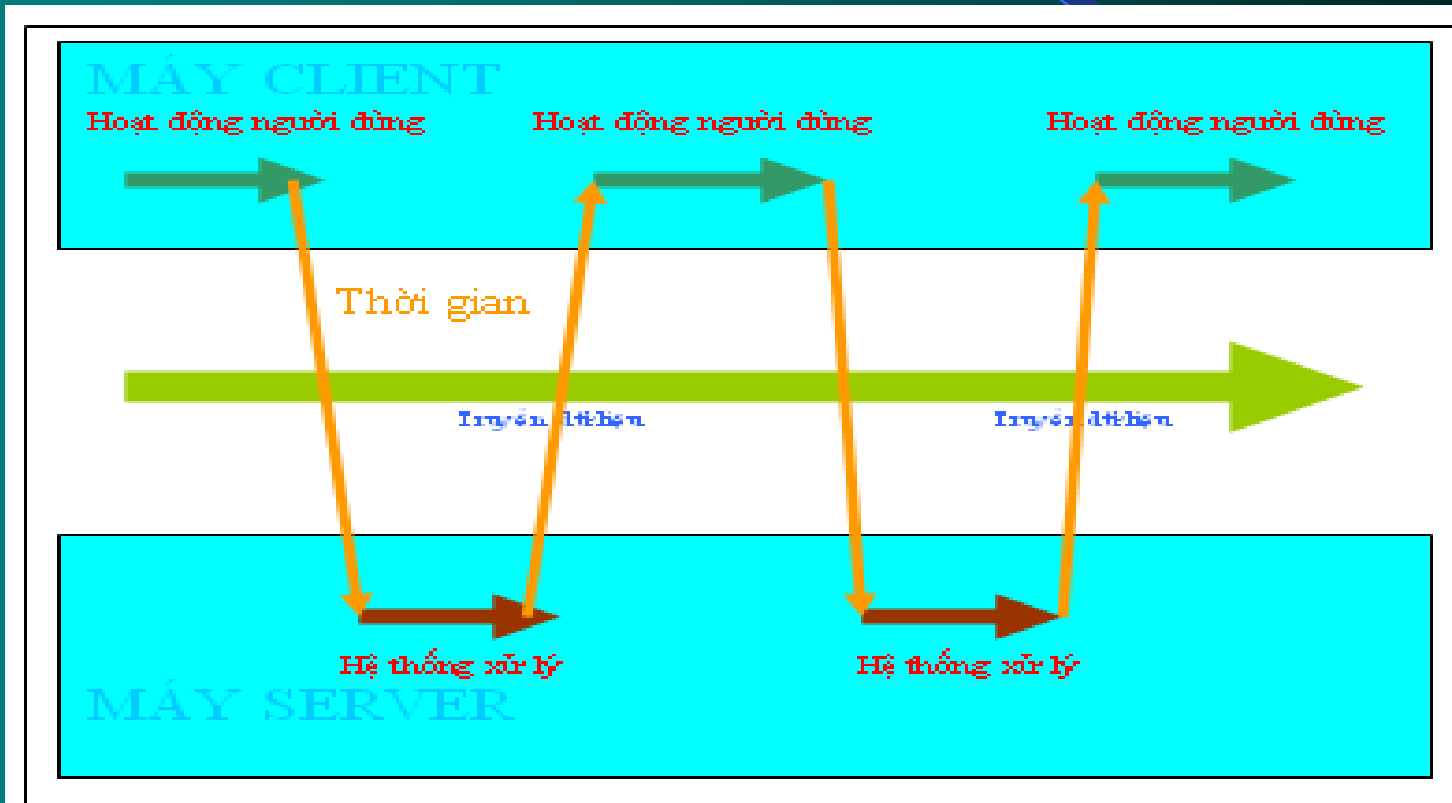
### *3. Mô hình tương tác Asynchronous*

- + Sự cải tiến trong mô hình tương tác máy chủ - máy khách là một trong những điểm khác biệt của Ajax
- + Mô hình tương tác của các ứng dụng web truyền thống (Mô hình đồng bộ )

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 3. Mô hình tương tác Asynchronous



Mô hình tương tác của các ứng dụng web truyền thống  
(Mô hình đồng bộ)

# CÔNG NGHỆ AJAX

## *I. Khái quát chung :*

### *3. Mô hình tương tác Asynchronous*

+ Theo mô hình trên có sự thiếu hợp lý ở chỗ: Khi người dùng có yêu cầu và gửi lên máy chủ thì chờ cho tới khi máy chủ xử lý xong và sau khi nhận được kết quả từ máy chủ trả về thì người dùng mới có thể tiếp tục thực hiện các thao tác tiếp theo.

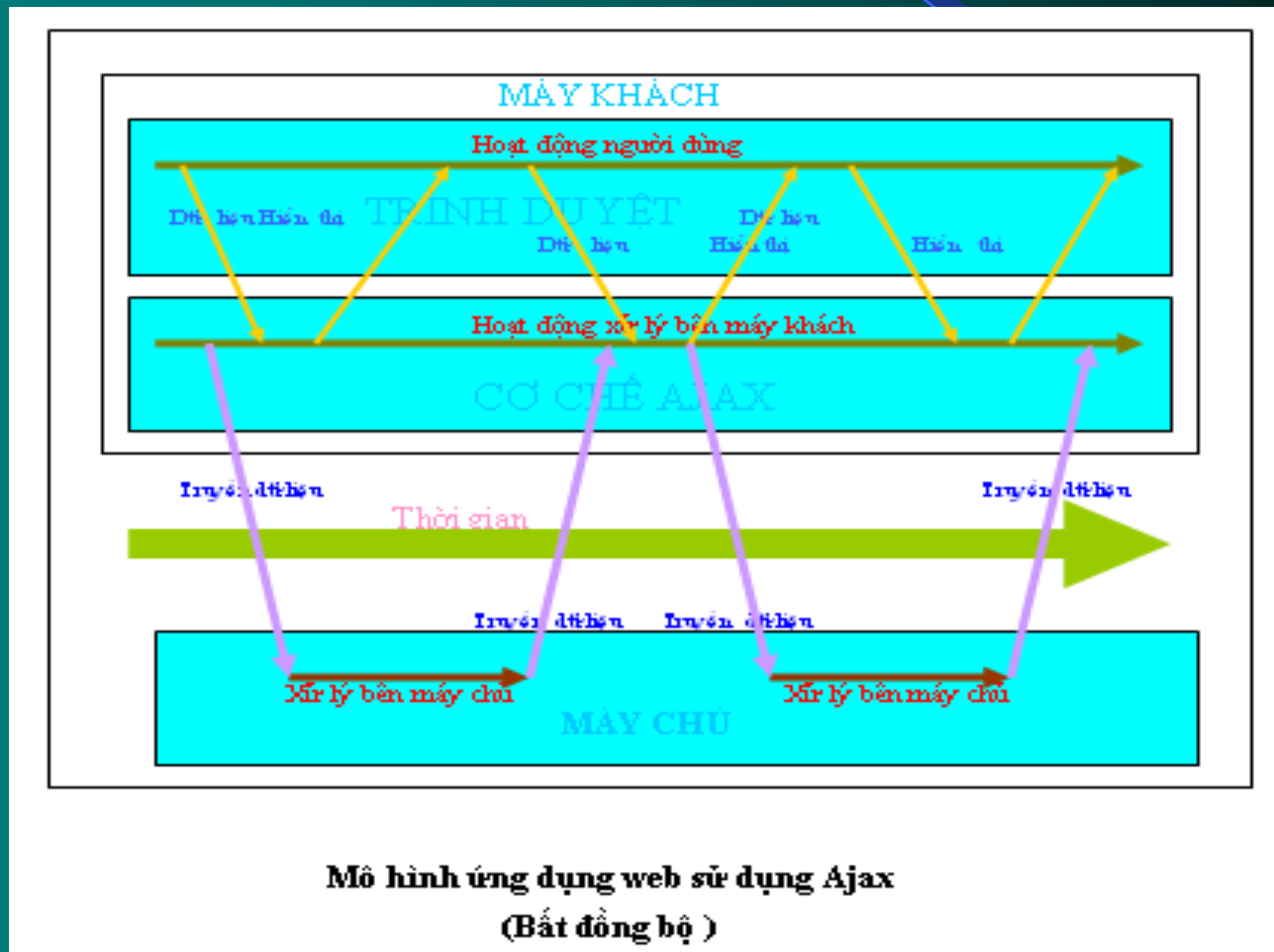
+ Như vậy thì đặt trong trường hợp nếu ta cần xử lý dữ liệu lớn thì không còn cách nào khác là phải chờ khá lâu.

+ Đây chính là một nhược điểm của các công nghệ Web truyền thống và Ajax đã giải quyết được vấn đề này .

# CÔNG NGHỆ AJAX

## I. Khái quát chung :

### 3. Mô hình tương tác Asynchronous



# CÔNG NGHỆ AJAX

## *I. Khái quát chung :*

### *3. Mô hình tương tác Asynchronous*

+ Để ý rằng tương tác giữa giao diện máy khách của chúng ta với Ajax diễn ra mang tính chất ngẫu nhiên và rất thường xuyên

+ Tương tác giữa Ajax và máy chủ xảy ra với chu kỳ dài hơn nhiều. Chính vì vậy việc xử lý những yêu cầu thực hiện trên máy chủ sẽ không ảnh hưởng gì đến quá trình tương tác giữa người dùng với máy client. Do đó làm giảm đi khoảng thời gian đợi như ở các mô hình truyền thống.

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *1. Ngôn ngữ XML*

+ XML (Extensible Markup Language) - là một bộ quy luật bao gồm các quy tắc để chia một tài liệu ra làm nhiều phần, rồi đánh dấu và ráp các phần khác nhau lại để dễ nhận diện chúng.

+ Tổ hợp Web toàn cầu W3C gọi XML là “một cú pháp thông dụng cho việc hiển thị cấu trúc trong dữ liệu”.

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *1. Ngôn ngữ XML*

- + Mục tiêu chính của XML là tách biệt nội dung và dạng biểu diễn của văn bản (Baldwin).

- + Cùng một nội dung ta có thể biểu diễn dưới nhiều hình thức khác nhau : chẳng hạn in ra giấy hoặc đưa lên Web, và có thể áp dụng nhiều thuộc tính trên văn bản như in đậm, in nghiêng, in gạch dưới, dùng màu,...

- + Ngoài ra nhờ việc sử dụng các ký tự thông thường mà con người có thể hiểu được nội dung của văn bản ngay khi còn đang ở dạng thô, chưa được áp dụng bất kỳ dạng biểu diễn nào

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *1. Ngôn ngữ XML*

Một trang XML cần phải theo đúng các quy luật sau đây :

- + Trang XML phải bắt đầu bằng câu tuyên bố XML
- + Mỗi bộ phận, gọi là “element” phải nằm giữa Tag Pair.
- + Nếu Tag nào không chứa gì ở giữa thì phải chấm dứt bằng”/>”

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *1. Ngôn ngữ XML*

Một trang XML cần phải theo các quy luật sau:

- + Một trang XML phải có một element độc nhất chứa tất cả các element khác. Đó là root của cây biểu diễn trang XML .

- + Các Tag Pair không được xen kẽ nhau, các Tag Pair phải đánh vần đúng y như nhau kể cả chữ hoa, chữ thường (<STUDENT> và </student> là bất hợp lệ) và các Attribute đều phải nằm giữa hai ngoặc kép (standalone=yes là bất hợp lệ, phải viết standalone="yes").

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *1. Ngôn ngữ XML*

Ví dụ :

```
<?xml version="1.0" standalone="yes"?>
```

```
<thegi>
```

*Chào XML ! Mạnh giỏi ?*

```
</thegi>
```

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 2. *Đối tượng DOM* (Document Object Model)

- + Mô hình đối tượng tài liệu là một giao diện lập trình ứng dụng (API).
- + Thông thường DOM có dạng một cây cấu trúc dữ liệu và được dùng để truy xuất các tài liệu HTML và XML.
- + Mô hình DOM hoạt động độc lập với hệ điều hành và dựa theo kỹ thuật lập trình hướng đối tượng để mô tả tài liệu.
- + Trong Ajax mô hình DOM đóng vai trò phân tách dữ liệu truyền dưới dạng XML.

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *2. Đối tượng DOM*

+ Để phân tích một văn bản XML bằng máy tính ta thường dùng một trong hai hoặc cả hai mô hình phân tích là SAX (Simple API for XML) và DOM.

+ Theo mô hình phân tích DOM, toàn bộ nội dung của một văn bản XML sẽ được lưu giữ vào bộ nhớ trong dưới dạng một cấu trúc cây

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *3. Giới thiệu về XHTML*

+ Trong Ajax XHTML cùng với CSS đóng vai trò hiển thị dữ liệu cho trang Web.

+ XHTML là một chuẩn kết hợp giữa HTML và XML và trở thành một định dạng riêng cho web. XHTML cho phép HTML được mở rộng bằng các thẻ sở hữu.

+ Ngoài ra XHTML cũng được mã hoá chặt chẽ hơn HTML và phải tuân thủ nhiều quy tắc cấu trúc hơn HTML

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *3. Giới thiệu về XHTML*

+ Trong XHTML tất cả các thẻ phải được đóng, các thuộc tính phải được cho vào dấu ngoặc kép (“”), và đặc biệt tất cả các thẻ và thuộc tính đều phải viết bằng chữ cái thường.

+ Mặc dù điều này có thể XHTML bị xem là cứng nhắc nhưng chính điều này lại là ưu điểm của nó: quy chuẩn và nguyên tắc.

+ XHTML là phiên bản kết hợp của HTML nên có những điểm khác cơ bản với HTML

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *3. Giới thiệu về XHTML*

- Các File XHTML cần phải well-formed có nghĩa là các thẻ <htm> <head> <body>...phải tuân thủ theo chuẩn .
- Các thẻ lồng nhau phải đúng trật tự .
- Các thẻ và các thuộc tính của các phần tử phải được viết bằng chữ thường.
- Các thuộc tính của các phần tử phải được đóng nháy kép: ví dụ <table border="1" width="100%">,...
- Các thẻ được gọi là noempty cần phải được đóng bằng một thẻ đóng( <br> ...</br>)

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 3. Giới thiệu về XHTML

- Không cho phép giản lược các thuộc tính.

Ví dụ : trong HTML có thể viết `<option selected>` nhưng trong XHTML phải viết đủ `<option selected="selected">` và không được đóng bằng thẻ `</option>`.

- Thuộc tính name được thay bằng thuộc tính id.

Ví dụ : `` được thay thế bằng `` - Phải khai báo DOCTYPE trước thẻ `<html>`

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### *3. Giới thiệu về XHTML*

```
<!DOCTYPE Doctype goes here>
```

```
<html>
```

```
 <head>
```

```
 <title>Ví dụ về XHTML</title>
```

```
 </head>
```

```
 <body>
```

```
 This is XHTML page
```

```
 </body>
```

```
</html>
```

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 4. CSS (*Cascading Style Sheets*)

- + Tập định kiểu theo tầng, được dùng để miêu tả cách trình bày các tài liệu viết bằng ngôn ngữ HTML, XHTML, XML, SVG, XUL...

- + Các thông số kỹ thuật của CSS do World Wide Web Consortium (W3C) quản lý. Trong Ajax CSS cùng với XHTML đóng vai trò hiển thị dữ liệu cho trang web .

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 5. DHTML

+ Hay HTML động tạo một trang Web bằng cách kết hợp các thành phần ngôn ngữ đánh dấu HTML tĩnh, ngôn ngữ lệnh máy khách (như JavaScript ) và ngôn ngữ định dạng CSS và DOM.

+ Do có khả năng phong phú, DHTML còn được sử dụng như một công cụ để xây dựng các trò chơi đơn giản trên trình duyệt

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 6. Đối tượng XMLHttpRequest

+ Để hiểu được cơ chế hoạt động của Ajax thì chúng ta phải biết được XMLHttpRequest là gì ? Như nhiều chuyên gia đã nhận định một trong những điểm quan trọng nhất của Ajax là nằm ở XMLHttpRequest

+ Điều đó cho thấy rằng, vì XMLHttpRequest có vai trò quan trọng đặc biệt trong công nghệ như vậy nên một ứng dụng của Ajax có thực thi và mang hiệu quả hay không phụ thuộc vào việc trình duyệt có hỗ trợ đối tượng này. Nếu một trình duyệt không hỗ trợ XMLHttpRequest sẽ không thể diễn ra quá trình trao đổi giữa máy client và máy server

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 6. Đối tượng XMLHttpRequest

+ Thông thường trong một ứng dụng web, để có thể gửi một HTTP Request đến máy chủ bằng JavaScript bạn cần tạo một đối tượng của lớp cung cấp tính năng này. Trong Internet Explorer (IE) thì lớp này tồn tại dưới dạng một ActiveX có tên là XMLHTTP

+ Đối tượng XMLHttpRequest không phải là một chuẩn của W3C (World Wide Web Consortium). Đối tượng XMLHttpRequest được hỗ trợ ở IE 5.0+, Safari 1.2, Mozilla1.0/Firefox, Opera 8.0 và Netscape 7+.

```
var httpRequest = new XMLHttpRequest();
```

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 6. Đối tượng XMLHttpRequest

+ Do sự khác biệt này nên để các ứng dụng có thể chạy được trên các trình duyệt chúng ta phải khai báo:

```
if(window.XMLHttpRequest)
{ //Mozilla,Safari ,..
 httpRequest=new XMLHttpRequest();
}else if(window.ActiveXObject)
{ //IE..
 httpRequest=new
ActiveXObject("Microsoft.XMLHTTP");
}
```

# CÔNG NGHỆ AJAX

## II. Các khái niệm liên quan Ajax :

### 6. Đối tượng XMLHttpRequest

+ Do ActiveX trên IE rất nguy hiểm cho nhiều người dùng nên trong nhiều trường hợp tính năng này bị cấp. Vì vậy bạn cần phải kiểm tra trình duyệt trước khi gọi đối tượng XMLHttpRequest. Quá trình kiểm tra này được thực hiện thông qua giá trị của window.ActiveXObject

```
Ví dụ : if(window.ActiveXObject) {
 xmlhttp=new
 ActiveXObject("Microsoft.XMLHTTP");
 }
 else{...}
```

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## I. Giới thiệu

Ngôn ngữ SQL (Structured Query Language) được hãng máy tính IBM đưa ra vào năm 1970.

Là một ngôn ngữ có cấu trúc, có nghĩa là bằng tập hợp các dòng lệnh sẽ chỉ cho hệ quản trị CSDL biết là người sử dụng sẽ làm gì chứ không chỉ ra làm như thế nào.

Ngôn ngữ SQL cung cấp một công cụ truy vấn (query) nhằm cho phép người sử dụng đặt câu hỏi với dữ liệu đang chứa trong các bảng, dữ liệu được rút ra từ những truy vấn có thể có nguồn từ một hay nhiều bảng khác nhau.

Sau đó tham khảo kết quả này, xem xét, phân tích nó kỹ lưỡng để có những xử lý thông tin thiết thực hơn.

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

Trong mục này trình bày một số lệnh cơ bản dùng để thao tác trên cơ sở dữ liệu như: chọn dữ liệu, thêm vào dữ liệu mới, xoá dữ liệu cũ, sửa đổi dữ liệu.

**1. Lệnh Select** : Lệnh này dùng để lấy dữ liệu trong một bảng thoả mãn điều kiện nào đó.

Cú pháp:

```
Select [*fields] From tablename
[Where expression]
[Group by fields1]
[Having expression1]
[Order by {colname } [ASC | DESC]]
```

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

- \* : tất cả các trường đều được chọn.
- *colname*: tên cột.
- *tablename*: là tên của một Table hoặc nhiều Table mà các Record được chọn.
- *expression*: là biểu thức điều kiện được đặt ra mà các bản ghi được chọn từ *tablename* phải thoả mãn biểu thức này.
- *expression1*: một biểu thức điều kiện đi sau mệnh đề Having. Mệnh đề Having thông thường đi kèm với mệnh đề Group by, *expression1* là biểu thức không tác động vào toàn bảng được chỉ ra bởi mệnh đề From mà chỉ tác động lần lượt từng nhóm các bản ghi đã được chỉ ra bởi mệnh đề Group by.
- Order by: dùng để sắp các bản ghi tăng hay giảm

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 1. Lệnh Select

- Ví dụ: Giả sử có một Table có tên NHANSU như sau:

Maso	Hoten	Ngaysinh	Nghenghiep
1	Lê Văn Tuấn	10/8/1990	Học sinh
2	Nguyễn Văn Hà	7/8/1970	Kỹ s
3	Nguyễn Thị Hoa	30/07/1980	Sinh viên

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 1. Lệnh Select

- Nếu sử dụng lệnh: **Select Hoten, Ngaysinh From nhansu**
- Sẽ chọn tất cả các bản ghi nhưng chỉ giới hạn bởi hai trường Hoten và Ngaysinh.
- Cụ thể gồm các bản ghi sau:

Lê Văn Tuấn	10/8/1990
Nguyễn văn Hà	7/8/1970
Nguyễn Thị Hoa	30/07/1980

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

**2. Lệnh Insert Into** : Lệnh này dùng để chèn thêm một Record dữ liệu mới vào cho bảng.

- Cú pháp:

**Insert Into *table*[(*field1*[,*field2*[,...]])]  
Values(*value1*[, *value2*[,...])**

- Trong đó:

*Table*: tên của Table cần bổ sung bản ghi

*field1, field2*: tên của các trường cần bổ sung dữ liệu

*value1, value2*: giá trị cần bổ sung ứng với các trường *field1, field2...*

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 2. Lệnh Insert Into

- Ví dụ:
- Giả sử ta có được Table Nhansu như trên, bây giờ bổ sung vào bản ghi mới:  

```
Insert Into Nhansu(Hoten, Ngaysinh, Nghenghiep)
Values("Nguyễn văn An","10/7/1995","Học sinh")
```
- Sau khi thực hiện lệnh này bảng Nhansu bây giờ như sau:

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 2. Lệnh Insert Into

Maso	Hoten	Ngaysinh	Nghenghiep
1	Lê Văn Tuấn	10/08/1990	Học sinh
2	Nguyễn văn Hà	07/08/1970	Kỹ s
3	Nguyễn Thị Hoa	30/07/1980	Sinh viên
4	Nguyễn văn An	10/07/1995	Học sinh

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

**3. Lệnh Update :** Lệnh này dùng để thay đổi dữ liệu cho các trường trong một bảng thỏa mãn điều kiện nào đó

- Cú pháp:

*Update table Set Field1=value1[,...] Where expression*

- Trong đó:

*table:* là tên của Table chứa bản ghi cần sửa đổi.

*Field1:* trường cần thay đổi dữ liệu

*Value1:* giá trị dữ liệu mới

*expression :* là biểu thức, mà các bản ghi được cập nhật phải thỏa mãn biểu thức này

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 3. Lệnh Update

- Ví dụ:
- Sửa đổi trường Nghenghiep có giá trị mới thành "CNVC" cho tất cả các record thoả mãn điều kiện là trường Ngaysinh="07/08/1970":

**Update Nhansu Set Nghenghiep="CNVC" Where Ngaysinh="07/08/1970"**

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 3. Lệnh Update

Table Nhansu bây giờ như sau:

Maso	Hoten	Ngaysinh	Nghenghiep
1	Lê Văn Tuấn	10/08/1990	Học sinh
2	Nguyễn văn Hà	07/08/1970	CNVC
3	Nguyễn Thị Hoa	30/07/1980	Sinh viên
4	Nguyễn văn An	10/07/1995	Học sinh

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 4. Lệnh Delete

Lệnh này dùng để xoá một hoặc nhiều record trong một bảng thoả mãn điều kiện nào đó.

- Cú pháp:

**Delete \* From *table* Where *expression***

- Trong đó:

*table*: là tên Table chứa các bản ghi cần xoá.

\*: tất cả các trường của bản ghi đều bị xoá.

*expression* : là tất cả các bản ghi được xoá phải thoả mãn biểu thức này.

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## II. Các câu lệnh SQL

### 4. Lệnh Delete

Ví dụ:

- Xoá tất cả các bản ghi trong Table Nhansu có trường Nghenghiep="Học sinh".
  - Delete \* From nhansu Where Nghenghiep="Học sinh"
- Bây giờ table Nhansu trở thành:

Maso	Hoten	Ngaysinh	Nghenghiep
2	Nguyễn văn Hà	07/08/1970	CNVC
3	Nguyễn Thị Hoa	30/07/1980	Sinh viên

# Ngôn ngữ Truy vấn dữ liệu (SQL)

Chú ý: Trong mệnh đề WHERE, các phép toán được sử dụng là:

Phép toán	Mô tả
=	So sánh bằng
<>	So sánh khác
>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng
BETWEEN	Nằm giữa một khoảng
LIKE	So sánh mẫu chuỗi

Lưu ý: Trong một số phiên bản của SQL, phép toán <> có thể được viết dưới dạng !=

# Ngôn ngữ Truy vấn dữ liệu (SQL)

## \* Sử dụng dấu nháy

Lưu ý rằng ở ví dụ trên ta đã sử dụng hai dấu nháy đơn (') bao quanh giá trị điều kiện 'Sandnes'.

SQL sử dụng dấu nháy đơn bao quanh các giá trị ở dạng chuỗi văn bản (text).

Nhiều hệ CSDL còn cho phép sử dụng dấu nháy kép ("). Các giá trị ở dạng số không dùng dấu nháy để bao quanh.

## + Với dữ liệu dạng chuỗi văn bản:

Câu lệnh đúng:

```
SELECT * FROM Persons WHERE FirstName = 'Tove'
```

Câu lệnh sai:

```
SELECT * FROM Persons WHERE FirstName = Tove
```

+ Với dữ liệu dạng số:

Câu lệnh đúng:

```
SELECT * FROM Persons WHERE Year > 1965
```

Câu lệnh sai:

```
SELECT * FROM Persons WHERE Year > '1965'
```

## \* **Phép toán điều kiện LIKE**

Phép toán LIKE được dùng để tìm kiếm một chuỗi mẫu văn bản trên một cột.

### Cú pháp :

Cú pháp của phép toán LIKE như sau:

```
SELECT tên_cột FROM tên_bảng
WHERE tên_cột LIKE mẫu
```

Một ký hiệu % có thể được sử dụng để định nghĩa các ký tự đại diện. % có thể được đặt trước và/hoặc sau mẫu.

## + Sử dụng LIKE

Câu lệnh SQL sau sẽ trả về danh sách những người có tên bắt đầu bằng chữ O:

```
SELECT * FROM SINHVIEN
WHERE ten LIKE 'O%'
```

Câu lệnh SQL sau sẽ trả về danh sách những người có tên kết thúc bằng chữ a:

```
SELECT * FROM sinhvien
WHERE ten LIKE '%a'
```

Câu lệnh SQL sau sẽ trả về danh sách những người có tên kết chứa chuỗi la:

```
SELECT * FROM sinhvien
WHERE ten LIKE '%la%'
```

Hết !!!